

L18 — Lab Data Engineering · guida agli script

Lab del sabato (📅, 4h): gli studenti mettono insieme **L16 (load) + L17 (clean)** su un **dataset volutamente sporco** (9 file), scoprono dove il parsing nativo non basta e imparano a usare **LlamaParse** da professionisti: prima la **caccia al tier** nel playground (fast 1 · cost_effective 3 · agentic 10 · agentic_plus 45 crediti/pagina), poi il loader via REST v2 dietro la stessa interfaccia **Document**, con la mappa di **routing** file → parser/tier. Il metodo del giorno: **carica → guarda l'output → diagnostica → correggi**. Niente notebook, solo script `.py`.

Il check base è **strutturale**: istantaneo e a zero crediti, si lancia a ogni salvataggio; il parse reale (loader LlamaParse, `check_lab18.py --live`) sta dietro un flag esplicito.

Ruolo di ogni file

Legenda: 🧰 ambiente/toolkit riusabile · 🔊 codice di riferimento che il docente live-coda · 📅 da completare dagli studenti · 📄 documenti/dati di esempio.

File	Ruolo	Cosa fa
<code>requirements.txt</code>	🧰	Dipendenze (loader L16/L17 + <code>httpx</code> / <code>dotenv</code> per LlamaParse; <code>reportlab</code> / <code>pillow</code> solo per rigenerare il dataset).
<code>.env.example</code>	🧰	Template per <code>LLAMA_CLOUD_API_KEY</code> (si crea in classe , free tier).
<code>CONSEGNA.md</code>	🧰	Promemoria operativo del lab per gli studenti.
<code>scaffolding/make_dataset.py</code>	🧰	(Ri)genera il dataset sporco: 9 file, trappole e tier diversi, tutti documentati nel docstring.
<code>dataset/guida_smartworking.pdf</code>	📄	Policy di 3 pagine , testo pulito ma con header/footer ripetuti e numeri di pagina: il caso “quasi facile” (il cleaning lavora comunque).
<code>dataset/listino_prezzi.pdf</code>	📄	2 pagine con tabella di 27 righe : pypdf estrae le celle alla rinfusa (TESTO ROTTO).

File	Ruolo	Cosa fa
<code>dataset/verbale_scansionato.pdf</code>		2 pagine “scansione” (grana, pagina storta) di solo testo : estrazione vuota — ma per l’OCR basta il tier <code>fast</code> .
<code>dataset/fattura_scansionata.pdf</code>		Fattura scansionata con tabella : OCR + struttura insieme — il <code>fast</code> legge il testo ma spappola la tabella.
<code>dataset/rassegna_stampa.pdf</code>		Ritaglio di giornale a 2 colonne scansionato: il <code>fast</code> mischia le colonne, dal <code>cost_effective</code> in su l’ordine torna.
<code>dataset/report_trimestrale.pdf</code>		PDF troncato : il loader solleva un’eccezione.
<code>dataset/faq_prodotto.html</code>		Pagina e-commerce vera: cookie banner, menu, breadcrumb, promo, sidebar, recensioni, newsletter, footer a colonne — tutto in <code><div></code> (sopravvive al loader L16 → tocca al cleaning).
<code>dataset/contratto_fornitura.docx</code>		Contratto di 8 articoli con 2 tabelle (SLA e corrispettivi): il loader legge solo i paragrafi, le tabelle spariscono in silenzio .
<code>dataset/note_interne.txt</code>		Testo in latin-1 : <code>UnicodeDecodeError</code> se letto utf-8.
<code>scaffolding/aikit/</code>		Il toolkit di L16–L17 (loaders + clean), invariato : oggi si usa, non si riscrive.
<code>working/primo_giro.py</code>		Demo guida : il canovaccio del “primo giro” (carica → guarda → diagnostica → correggi su <code>faq_prodotto.html</code>) che il docente scrive dal vivo; versione finita in <code>solutions/</code> .
<code>working/triage.py</code>		Lab 1 : caricare tutto il dataset, gestire gli edge case, classificare OK/VUOTO/ERRORE, scrivere <code>triage.md</code> (TODO 1–3).

File	Ruolo	Cosa fa
<code>working/llamaparse.py</code>		Il loader gestito col parametro <code>tier</code> : flusso REST v2 (files → parse col tier → poll), via <code>httpx</code> puro (TODO 4, si scrive col docente).
<code>working/scheda_tier.md</code>		La scheda della caccia al tier : per ogni file candidato, tier provati, crediti, difetto risolto?, tier scelto e perché. Si compila nel playground. (Unico file di <code>working/</code> senza controparte in <code>solutions/</code> , per scelta: la “soluzione” è la vostra sperimentazione.)
<code>working/pulisci_corpus.py</code>		Lab 2 : lo script riproducibile con la mappa ROUTING file → nativo/tier (TODO 5). Il cleaning di L17 è fornito . Scrive <code>corpus_pulito/</code> + <code>report.md</code> con tier e crediti per file.
<code>scaffolding/check_lab18.py</code>		Verifica <code>[OK]/[FAIL]</code> per requisito — istantanea, zero crediti , da lanciare a ogni salvataggio; <code>--live</code> fa un parse vero del listino (≈6 crediti).
<code>solutions/</code>		Le versioni complete dei soli file di <code>working/</code> (stesso nome, stessi import): quelle che il docente mostra a fine blocco.

Setup

```
cd scripts
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env          # la chiave LlamaParse si crea in classe
python scaffolding/make_dataset.py  # opzionale: il dataset è già versionato
```

Nota per chi distribuisce (docente): agli studenti la cartella arriva **senza** `.env` e `.venv` (git non li versiona). Se la zippi a mano, escludili: un `.env` con una chiave dentro contraddice il “la chiave si crea in classe” — e fa consumare crediti veri al primo `pulisci_corpus.py`.

Come si prova

Tutto si lancia dalla cartella `scripts/`:

```
python working/triage.py           # Lab 1: tabella di triage + triage.md
python working/pulisci_corpus.py   # Lab 2: corpus_pulito/ + report.md
    (offline)
python scaffolding/check_lab18.py  # verifica working/: istantanea, zero
    crediti
python scaffolding/check_lab18.py --live # parse vero via LlamaParse (~6 crediti)
python scaffolding/check_lab18.py --solutions # per il docente: verifica solutions/
```

Nota: leggendo il PDF corrotto, `pypdf` stampa su stderr l'avviso `EOF marker not found` — è atteso, fa parte della diagnosi.

Le idee da trasmettere

1. **“Esce del testo” ≠ “esce il testo giusto”.** Il listino e il contratto si caricano senza errori, ma la tabella è distrutta o sparita: il triage serve a guardare, non solo a “non crashare”. (È l'anteprima nella tabella di triage a smascherarli.)
2. **Un servizio esterno si innesta dietro la TUA interfaccia.** LlamaParse entra come `load_llamaparse(path, tier=...) → list[Document]`: il resto della pipeline non sa chi ha fatto il parsing. Niente SDK: è solo un'API REST (files → parse col tier → poll), tre chiamate `httpx`.
3. **Il tier giusto è il più basso che basta.** Verificato sul nostro dataset: il verbale (scansione di solo testo) si risolve col `fast` a 1 credito/pagina; fattura, rassegna, listino e contratto chiedono il `cost_effective` (3); l' `agentic` (10) esiste per ciò che resta rotto — e va provato, non comprato a scatola chiusa. La scheda dei tier serve a deciderlo coi numeri.
4. **Il trade-off locale vs gestito.** Nativo: gratis, offline, privato, ma sul difficile (tabelle, scansioni, colonne) perde. Gestito: robusto, ma a crediti (free tier ~10k/mese), più lento (job asincrono) e i **documenti viaggiano nel cloud**. Si sceglie per file e per tier, non per fede — e il `report.md` con loader e crediti lo dimostra coi numeri.
5. **Irrecuperabile esiste.** Il PDF troncato non lo salva nessun parser (a nessun tier): la risposta giusta è diagnosticarlo e chiederne una copia sana, non insistere.

E l'SDK ufficiale? (domanda legittima in aula)

Stato verificato a luglio 2026, con parse reali:

- `llama-cloud-services` (quello citato da quasi tutti i tutorial) è **deprecato** (manutenzione terminata a maggio 2026) e llama-index lì è **inscindibile**: `llama-index-core` è dipendenza hard e i metodi restituiscono oggetti `llama_index.core.schema.Document` (66 pacchetti, ~240 MB di venv).

- Il successore `llama-cloud ≥ 1.0` è invece un client **pulito**: 17 pacchetti, ~38 MB, dipende solo da `httpx` + `pydantic` — zero llama-index. Una chiamata fa tutto:
`client.parsing.parse(tier="cost_effective", version="latest", upload_file=f, expand=["markdown"])` (upload + poll + risultato).
- **Nel lab restiamo sul REST a mano**: le tre chiamate mostrano il job asincrono, che è il punto didattico. In produzione il nuovo SDK va benissimo — e dopo il lab gli studenti sanno esattamente cosa fa sotto.

Percorso in aula (le slide sono la regia: ogni switch ha la sua slide)

1. **Demo guida (docente, 🎤)**: il “primo giro” su `faq_prodotto.html` con `primo_giro.py` — load, ispezione (4515 char di cui i primi 400 di titolo e cookie banner), diagnosi, primo fix mirato. I file che si rompono restano da scoprire agli studenti.
2. **Lab 1 (studenti, 🖥️)**: `triage.py` su tutto il dataset (TODO 1–3).
3. **LlamaParse webapp (🌐🎤)**: registrazione in classe + demo del giro nel playground (tier, markdown, crediti in dashboard).
4. **Caccia al tier (studenti, 🖥️🌐)**: i 5 file candidati su ≥2 tier ciascuno nel playground → `scheda_tier.md`.
5. **Loader via codice (🎤, si replica)**: `llamaparse.py` con il parametro `tier` (TODO 4).
6. **Lab 2 (studenti, 🖥️)**: la mappa `ROUTING` (TODO 5) → `corpus_pulito/` + `report.md` con tier e crediti.
7. **Show & tell (📊)**: schede e report a confronto; il tier che ripaga e quello che no; trade-off finale.